Vse za učenje APS2

Inventar

Inventar gradiva za izpit APS2

Delovna mapa: `D:\Šola\FRI\SecondYear\Alogirtmi\Izpit`

Struktura

- `Predavanja/` - 8 PDF datotek s prosojnicami.
- `Vaje/` - 13 PDF datotek z nalogami iz vaj.
- `Primeri/` - 13 slik primerov testov oziroma izpitnih pol.
- `results/` - mapa za pripravljene rešitve in učne povzetke.

Predavanja

Datoteka	Strani	Prepoznana tema
`eaa263f7-76f0-4079-ad23-58eeea71b9a7.pdf`	8	Uvod v algoritme in podatkovne strukture, časovna zahtevnost
`c9a3298c-761d-49b8-8af4-7cbe31c6282c.pdf`	34	ADT, tabele, slovarji, seznami, preskočni seznami
`b8a0228c-5230-4ccc-be50-eee727f2bc1c.pdf`	17	Amortizirana analiza
`9789321c-64c3-45f0-a279-56d205df82ce.pdf`	53	Drevesa: dvojiška, iskalna, AVL, večsmerna, B/B+, TTF, rdeče-črna
`41e289b2-60c5-48f4-b5f7-1599164cb827.pdf`	34	Vrsta s prednostjo, zgoščene tabele, Bloomov filter, disjunktne množice
`4d7fa5ce-45de-4397-833e-bfead45e6b22.pdf`	30	Dinamično programiranje, verižno množenje matrik, nahrbtnik
`f9097b46-0b19-48a1-a360-83f8ae291bd8.pdf`	14	Razredi kompleksnosti, P/NP, redukcije
`2a7980df-6c3e-4fe2-aad2-fe12d98d310f.pdf`	42	Grafi: MST, barvanje, najkrajše poti, pretoki

Opomba za grafe: po navodilu obdelam samo Prim, Kruskal, topološko urejanje, Bellman-Ford, Dijkstra in Welsh-Powell. Največji pretok preskočim.

Vaje

Datoteka	Strani	Prepoznana vsebina
`b53d52a2-70e1-44c6-8a18-9a7b62821c3f.pdf`	2	Asimptotika, dokazovanje Θ/O, kopice
`0ffa67ac-647d-4550-bb25-a680dc598532.pdf`	1	Največje podzaporedje, osnovni algoritmi
`9770dd55-2718-4f70-ac34-2d59ff7bae55.pdf`	1	Amortizirana analiza sklada
`45d204a3-f745-457d-8df5-c302eb96e01d.pdf`	1	Java `List`, implementacije ADT
`c8049133-4b00-4352-81aa-4cdfd0330146.pdf`	2	Povezani seznami, obračanje seznama, preskočni seznami
`5689f7ec-5e26-42ba-99a0-793ba9ec6b15.pdf`	2	BST, min-kopica, izpisi, implementacija dreves
`b2990eb4-f36c-4dc1-8700-3972034d955d.pdf`	1	AVL vstavljanja/brisanja in rotacije
`4d82ebc6-7d38-43f5-ab39-0f85ed9d115f.pdf`	1	Zgoščene tabele, trki, naslavljanje

Datoteka	Strani	Prepoznana vsebina
`7f6c1956-6ae3-4b18-a72d-f8618f87cf3c.pdf`	1	0/1 nahrbtnik, dinamično programiranje
`89257b34-30a5-4be6-b8c3-6c640ee4bc11.pdf`	1	Vertex cover, kompleksnost
`8c28068f-2372-41bc-80be-cf5a94e52bcc.pdf`	1	Verižno množenje matrik
`f9310514-5098-4f50-b2ed-dc43d8277a0c.pdf`	1	Grafi: MST/najkrajše poti na uteženem neusmerjenem grafu
`f4daa6ad-596e-4de1-a2a5-75b4196ab4a0.pdf`	2	Grafi: najkrajše poti, topološko urejanje, štetje poti

Primeri

Slike v `Primeri/`:

- `20240610_100725.jpg`
- `20240620_100452.jpg`
- `20240620_100530.jpg`
- `20240620_100533.jpg`
- `20240620_100553.jpg`
- `image0.jpg`
- `IMG_5392.jpg`
- `IMG_5395.jpg`
- `IMG_5396.jpg`
- `PXL_20230619_080959115.jpg`
- `PXL_20230619_081001816.jpg`
- `PXL_20230619_081016189.jpg`
- `PXL_20230619_081022358.jpg`

Tehnična opomba

- Originalnih PDF-jev ne spreminjam.
- Na PATH-u ni bilo najdenega `pdftotext`, `pdftoppm` ali `tesseract`.
- Uporabljen je priložen Codex Python runtime z `pypdf`, `PIL` in `reportlab`.
- Ker varno neposredno dopolnjevanje PDF-jev ni zagotovljeno, bodo rešitve praznih delov iz predavanj pripravljene kot spremljevalna datoteka v `results/`.

Rešitve predavanj

Rešitve in dopolnitve praznih mest iz predavanj

Opomba: originalnih PDF-jev nisem spreminjal. Ker direktno urejanje PDF-jev brez zanesljivega renderiranja ni varno, so spodaj rešitve organizirane po datoteki, temi in strani/prosojnici. To uporabi kot spremljevalni dokument ob prosojnicah.

`eaa263f7-76f0-4079-ad23-58eeea71b9a7.pdf` - uvod, zahtevnost

Stran 3 - zakaj merimo čas teoretično

- Čas izvajanja nas zanima, ker želimo primerjati algoritme neodvisno od konkretnega računalnika.
- Razlika med 1 ms in 2 ms postane pomembna pri velikem številu ponovitev ali velikih vhodih.
- V teoriji časa ne merimo v sekundah, ampak v številu osnovnih korakov kot funkcijo velikosti vhoda `n`.
- Če je `T(n)=C*f(n)`, pri asimptotiki konstante in nižje člene ignoriramo.

Strani 6-8 - kako veliko je veliko

Za grobo oceno velja:

- `O(log n)` in `O(n)` sta običajno zelo dobra.
- `O(n log n)` je praktičen za velika urejanja.
- `O(n^2)` hitro postane problem pri milijonih elementov.
- `O(2^n)` in `O(n!)` sta uporabna samo za zelo majhne `n`.

Mini trik: če v nalogi piše 1 GHz, računaj približno `10^9` osnovnih operacij/s.

`c9a3298c-761d-49b8-8af4-7cbe31c6282c.pdf` - ADT in osnovne strukture

Stran 5 - dinamična tabela

Smiselno povečevanje tabele: ob zapolnitvi povečamo kapaciteto za faktor, najpogosteje `2x`.

Zakaj ne povečujemo za `+1`?

- Pri `+1` bi ob skoraj vsakem vstavljanju kopirali celo tabelo: skupno `1+2+...+n = Θ(n^2)`.
- Pri podvajanju je skupno število kopiranj do `n` vstavljanj `O(n)`, zato je amortizirana cena `insert` enaka `O(1)`.

Stran 8 - tabela tabel

Za `n` elementov potrebujemo tabele velikosti `1, 2, 4, 8, ...`, podobno kot binarni zapis števila `n`.

- Zasedena je tabela `A_i`, če je bit `i` v binarnem zapisu `n` enak 1.
- Število zasedenih tabel je `popcount(n)`.
- Največji indeks je `⌊log2 n⌋`, zato je vseh možnih nivojev do največjega `O(log n)`.
- Iskanje: v vsaki zasedeni urejeni tabeli uporabimo binarno iskanje.
- Čas iskanja: `O(log^2 n)`, ker je tabel `O(log n)`, vsaka pa se išče z bisekcijo.

- Vstavljanje je kot pri binarnem števcu: če je `A_i` polna, jo zlijemo s prenosom v `A_{i+1}`. Amortizirano `O(log n)` ali bolje, odvisno od modela štetja kopiranj.

Stran 13 - slovar

- Če dvakrat vstavimo isti ključ, slovar tipično posodobi vrednost, ne shrani dveh kopij istega ključa.
- Slovar ima lahko samo ključe; takrat je praktično množica.

Strani 21-31 - preskočni seznam

Preskočni seznam je urejen seznam z dodatnimi kazalci za preskoke.

Postopek iskanja:

1. Začni na najvišjem nivoju v glavi seznama.
2. Dokler naslednji element ni prevelik, se pomikaj desno.
3. Ko ne moreš več desno, pojdi en nivo dol.
4. Končaš na nivoju 0.

Naključna višina elementa:

- Element je vedno na nivoju 0.
- Mečemo kovanec; dokler pada izbrani uspeh, element promoviramo za en nivo.
- Pri verjetnosti `1/2` je pričakovana višina `O(log n)`.
- Pričakovano iskanje in vstavljanje: `O(log n)`.

`b8a0228c-5230-4ccc-be50-eee727f2bc1c.pdf` - amortizirana analiza

Metoda vsote

Za zaporedje n operacij izračunamo skupno najslabšo ceno `T(n)`. Amortizirana cena ene operacije je `T(n)/n`.

Primer razširjen sklad:

- Vsak element je lahko enkrat dodan in največ enkrat odstranjen.
- Če naredimo n operacij, je skupno število realnih premikov elementov `O(n)`.
- Amortizirana cena `push`, `pop`, `multipop`: `O(1)`.

Primer dvojiški števec:

- Bit `A[0]` se spremeni pri vsaki operaciji.
- Bit `A[1]` pri vsaki drugi.
- Bit `A[i]` pri vsaki `2^i`-ti.
- Skupno število sprememb bitov je `n + n/2 + n/4 + ... < 2n`.
- Amortizirana cena povečanja: `O(1)`.

Metoda kopičenja

Ideja: poceni operacije plačajo malo več in kredit shranimo za drage operacije.

- Sklad: `push` plača 2 enoti; 1 za vstavljanje, 1 kredit za kasnejši izbris.
- Dvojiški števec: ob nastavitvi bita na 1 shranimo kredit za kasnejši preklop nazaj na 0.

Metoda potenciala

Amortizirana cena:

```
amortizirana = realna cena + Φ(D_i) - Φ(D_{i-1})
```

Pazi: funkcija potenciala mora biti izbrana tako, da ne ustvarjamo negativnega dolga. Tipično zahtevamo `Φ(D_i) >= Φ(D_0)`.

`9789321c-64c3-45f0-a279-56d205df82ce.pdf` - drevesa

BST

- Iskanje/vstavljanje/brisanje: `O(h)`, kjer je h višina drevesa.
- Če je drevo uravnoteženo: `h=O(log n)`.
- Če je degenerirano v seznam: `h=O(n)`.
- Minimum v BST je najbolj levo vozlišče, maksimum najbolj desno.

AVL

AVL pogoj: za vsako vozlišče je razlika višin levega in desnega poddrevesa največ 1.

Rotacije:

- LL: desna rotacija.
- RR: leva rotacija.
- LR: leva rotacija nad levim sinom, nato desna rotacija.
- RL: desna rotacija nad desnim sinom, nato leva rotacija.

Čas: iskanje, vstavljanje, brisanje `O(log n)`.

B/B+ drevesa

- Uporabna za diske in baze, ker zmanjšajo število dostopov do blokov.
- Več ključev v vozlišču pomeni manjšo višino.
- B+ drevo hrani vse podatke v listih, notranja vozlišča so indeks.

Rdeče-črno drevo

Lastnosti:

- Vsako vozlišče je rdeče ali črno.
- Koren je črn.
- Rdeče vozlišče nima rdečega otroka.
- Vsaka pot od vozlišča do praznega lista ima enako število črnih vozlišč.

Čas: `O(log n)`, ker je višina omejena z `O(log n)`.

AVL je strožje uravnoteženo; RČ ima pogosto manj rotacij pri vstavljanju/brisanjih.

`41e289b2-60c5-48f4-b5f7-1599164cb827.pdf` - prioritetna vrsta, zgoščevanje, Bloom, DSU

Vrsta s prednostjo / kopica

- `min`: `O(1)` pri min-kopici.
- `insert`: `O(log n)` zaradi popravljanja navzgor.
- `deleteMin`: `O(log n)` zaradi popravljanja navzdol.
- `heapify` na celotni tabeli: `O(n)`, ne `O(n log n)`.

Zgoščena tabela

Veriženje:

- Indeks: `h(k)`.
- V košu hranimo seznam/BST elementov z istim indeksom.
- Pri dobri zgoščevalni funkciji in faktorju zasedenosti $\alpha=n/m$ je pričakovano iskanje $O(1+\alpha)$.

Odprto naslavljanje:

- Linearno: $h(k,i)=(h(k)+i) \bmod m$.
- Kvadratno ali dvojno zgoščevanje zmanjšata grupiranje.
- Brisanje potrebuje poseben označevalec `DELETED`, drugače prekinemo iskalne verige.

Bloomov filter

Odgovora sta:

- "zagotovo NE"
- "verjetno DA"

Lažno pozitiven odgovor je možen, lažno negativen pri navadnem Bloomovem filtru ni možen, če ne brišemo.

Brisanje:

- Navaden Bloomov filter ne zna varno brisati.
- Rešitev: counting Bloom filter s števci.

Disjunktne množice

Operacije:

- `makeSet(x)`
- `find(x)`
- `union(x,y)`

Učinkovita izvedba:

- gozd dreves,

- union by rank/size,
- path compression.

Amortizirano skoraj konstantno: $O(\alpha(n))$, kjer je α inverzna Ackermannova funkcija.

4d7fa5ce-45de-4397-833e-bfead45e6b22.pdf - dinamično programiranje

Fibonacci

Rekurzija brez shranjevanja:

- $T(n)=T(n-1)+T(n-2)+O(1)$
- eksponentno, približno $O(\phi^n)$.

Memoizacija ali tabulacija:

- vsako stanje izračunamo enkrat,
- čas $\Theta(n)$,
- prostor $\Theta(n)$ oziroma $\Theta(1)$, če hranimo samo zadnji dve vrednosti.

Verižno množenje matrik

Če ima M_i dimenzijo $d_{i-1} \times d_i$, je cena delitve pri k:

$$c(i,k) + c(k+1,j) + d_{i-1} * d_k * d_j$$

Rekurenca:

$$c(i,j)=\min_{i\leq k<j}(c(i,k)+c(k+1,j)+d_{i-1}d_kd_j)$$

Osnova: $c(i,i)=0$.

Čas: $O(n^3)$, prostor: $O(n^2)$.

0/1 nahrbtnik

$dp[i][w]$ = najboljša vrednost s prvimi i predmeti in kapaciteto w.

Če predmeta i ne vzamem:

$$dp[i][w]=dp[i-1][w]$$

Če ga vzamem:

$$dp[i][w]=dp[i-1][w-w_i]+v_i$$

Vzamem maksimum. Čas $O(nV)$ oziroma $O(nW)$, kjer je W kapaciteta.

f9097b46-0b19-48a1-a360-83f8ae291bd8.pdf - razredi kompleksnosti

- P: odločitveni problemi, rešljivi v polinomskem času.

- NP: rešitve znamo preveriti v polinomskem času.
- NP-poln: v NP in vsak problem iz NP se nanj polinomsko reducira.
- NP-težak: vsaj tako težak kot NP-polni, ni nujno v NP.

Pazi:

- Zahtevnost algoritma ni isto kot zahtevnost problema.
- Če poznamo eksponenten algoritem, to še ne pomeni, da problem ni v P.
- Če bi našli polinomski algoritem za en NP-poln problem, bi veljalo `P=NP`.

`2a7980df-6c3e-4fe2-aad2-fe12d98d310f.pdf` - grafi

Obdelane samo zahtevane teme: Prim, Kruskal, topološko urejanje, Bellman-Ford, Dijkstra, Welsh-Powell. Največji pretok je po navodilu izpuščen.

Prim

Uporabiš za MST v povezanem neusmerjenem uteženem grafu.

Postopek:

1. Začni v poljubnem vozlišču.
2. Vzdržuj množico že vključenih vozlišč.
3. Vedno dodaj najcenejšo povezavo iz drevesa do vozlišča zunaj drevesa.
4. Končaj po `n-1` povezavah.

Zahtevnost:

- matrika: `O(n^2)`,
- seznam sosedov + prioritetna vrsta: `O((n+m) log n)`.

Kruskal

Postopek:

1. Uredi povezave po teži.
2. Začni s praznim gozdom.
3. Povezavo dodaj, če ne ustvari cikla.
4. Za preverjanje komponent uporabi disjunktne množice.

Zahtevnost: `O(m log m)` zaradi urejanja povezav.

Topološko urejanje

Uporabiš samo za usmerjene aciklične grafe (DAG).

Kahnov algoritem:

1. Izračunaj vhodne stopnje.
2. V vrsto daj vozlišča z vhodno stopnjo 0.
3. Odstranjuj vozlišča in zmanjšuj stopnje naslednikov.

4. Če ne odstraniš vseh vozlišč, graf ima cikel.

Najkrajše poti v DAG:

1. Topološko uredi.
2. `d[start]=0`, ostalo ∞.
3. Sproščaj povezave v topološkem vrstnem redu.

Čas `O(n+m)`.

Dijkstra

Uporabiš, ko so vse uteži nenegativne.

Postopek:

1. `d[s]=0`, ostalo ∞.
2. Vedno izberi neobiskano vozlišče z najmanjšo trenutno razdaljo.
3. Sprosti njegove izhodne povezave.
4. Ko je vozlišče dokončno izbrano, se njegova razdalja ne bo več izboljšala.

Pazi: če vidiš negativno utež, Dijkstra odpade.

Bellman-Ford

Uporabiš pri negativnih utežeh ali ko želiš zaznati negativen cikel.

Postopek:

1. `d[s]=0`, ostalo ∞.
2. `n-1` krat sprosti vse povezave.
3. Še enkrat preveri vse povezave: če se kaj še izboljša, obstaja dosegljiv negativen cikel.

Čas `O(nm)`.

Welsh-Powell

Hevristika za barvanje grafa.

Postopek:

1. Vozlišča uredi padajoče po stopnji.
2. Vzemi prvo nepobarvano vozlišče in mu dodeli novo barvo.
3. Z isto barvo pobarvaj čim več naslednjih vozlišč, ki niso sosednja že pobarvanim s to barvo.
4. Ponavljaj, dokler niso pobarvana vsa vozlišča.

Rezultat ni nujno optimalen, je pa hiter in pogosto dober.

Rešitve primerov

Rešitve primerov iz mape `Primeri`

Opomba: slike so fotografije, zato so nekateri robovi nalog slabše berljivi. Kjer je potrebno, sem zapisal predpostavko.

`image0.jpg` - 2. izpitni rok 2024, 1. naloga

Naloga

Podani so trije algoritmi:

- A: urejanje z zlivanjem.
- B: najcenejša pot v usmerjenem polnem grafu z n točkami s topološkim urejanjem.
- C: osnovno množenje matrik.

a) Asimptotske zahtevnosti

- `T_A = Θ(n log n)` - merge sort razdeli tabelo na polovice in na vsakem nivoju opravi `Θ(n)` zlivanja.
- `T_B = Θ(n^2)` - poln graf ima `m=Θ(n^2)` povezav, topološko urejanje in sproščanje povezav porabita `Θ(n+m)=Θ(n^2)`.
- `T_C = Θ(n^3)` - osnovno množenje dveh `n x n` matrik ima tri gnezdene zanke.

b) Kolikšen `n` rešimo v 1 sekundi

Predpostavka: 1 GHz pomeni približno `10^9` osnovnih operacij/s, konstanta `C=2`, zato mora veljati `f(n) <= 5*10^8`.

- `n log2 n <= 5*10^8` -> `n ≈ 20.580.553`.
- `n^2 <= 5*10^8` -> `n ≈ 22.360`.
- `n^3 <= 5*10^8` -> `n ≈ 793`.

Pazi: to je groba teoretična ocena, ne realna meritev.

`IMG_5396.jpg` - 1. izpitni rok 2024, 1. naloga

Naloga

Oceniti čas urejanja imen vseh prebivalcev Kitajske, približno `n = 1.412.000.000`, za:

- kvadratični algoritem,
- algoritem `O(n log n)`.

Predpostavka: 1 osnovna operacija traja približno 1 ns, primerjava imen ima konstantno ceno.

Kvadratično urejanje

`n^2 = (1.412*10^9)^2 ≈ 1.993*10^18` operacij.

Pri `10^9` operacij/s:

`1.993*10^18 / 10^9 = 1.993*10^9 s`.

To je približno `63 let`.

`n log n` urejanje

`log2(1.412*10^9) ≈ 30.4`.

`n log2 n ≈ 1.412*10^9 * 30.4 ≈ 4.29*10^10` operacij.

Pri `10^9` operacij/s je to približno `43 sekund`.

Zaključek: razlika med `n^2` in `n log n` pri milijardnih vhodih ni “malo počasneje”, ampak razlika med minuto in življenjem.

`PXL_20230619_081001816.jpg` - stopnice

A) Samo koraki dolžine 1

Za n stopnic obstaja samo en način:

`1+1+...+1`.

Odgovor: `1`.

B) 4 stopnice, koraki 1 ali 2

Možnosti:

- `1+1+1+1`
- `1+1+2`
- `1+2+1`
- `2+1+1`
- `2+2`

Odgovor: `5`.

C) 5 stopnic, koraki 1 ali 2

Rekurenca:

`f(n)=f(n-1)+f(n-2)`

`f(0)=1`, `f(1)=1`, `f(2)=2`, `f(3)=3`, `f(4)=5`, `f(5)=8`.

Odgovor: `8`.

D) Splošna formula

`f(n)=f(n-1)+f(n-2)`, z osnovama `f(0)=1`, `f(1)=1`.

Razlaga: zadnji korak je ali dolžine 1, zato prej stojimo na `n-1`, ali dolžine 2, zato prej stojimo na `n-2`.

E) Učinkovit program

```
stStopnic(n):
    if n == 0 or n == 1:
        return 1
    a = 1
    b = 1
    for i = 2..n:
        c = a + b
        a = b
        b = c
    return b
```

Čas $\Theta(n)$, prostor $\Theta(1)$.

20240620_100553.jpg - tabela tabel

a) Koliko tabel potrebujemo za 2024 elementov

$2024 = 1024 + 512 + 256 + 128 + 64 + 32 + 8$

Binarno so zasedene tabele:

$A_{10}, A_9, A_8, A_7, A_6, A_5, A_3$

Odgovor: 7 zasedenih tabel.

Opomba: če bi šteli vse nivoje od A_0 do A_{10}, bi bilo nivojev 11, vendar je po strukturi “tabela tabel” običajno mišljeno število zasedenih tabel.

b) Koliko elementov lahko dodamo, da se število uporabljenih tabel ne spremeni

Če “uporabljene tabele” pomeni zasedene tabele, potem že pri dodajanju enega elementa:

$2024 + 1 = 2025$

se vklopi še bit A_0, zato se število zasedenih tabel spremeni. Odgovor: 0.

Če bi naloga mislila število nivojev do največje tabele (A_0 do A_{10}), lahko dodamo do $2047 - 2024 = 23$ elementov, preden potrebujemo A_{11}.

c) Časovna zahtevnost iskanja

Zasedenih je $O(\log n)$ urejenih tabel. V vsaki iščemo z bisekcijo.

Skupaj:

$O(\log n + \log n + \dots) = O(\log^2 n)$.

d) Vstavljanje v strukturo

Začetek:

- $A_0 = [9]$
- $A_1 = [7, 8]$

Dodamo $1, 5, 6, 2, 4, 3, 0, 10$.

Postopek je kot binarni števec: če je trenutna tabela zasedena, jo zlijemo s prenosom naprej.

Končno stanje:

- `A_0 = [10]`
- `A_1 = [0, 3]`
- `A_2 = []`
- `A_3 = [1, 2, 4, 5, 6, 7, 8, 9]`

Preverjanje: skupaj `1 + 2 + 8 = 11` elementov, kar ustreza začetnim 3 + dodanim 8.

`PXL_20230619_081016189.jpg` - minimalno vpeto drevo

A) Matrika cen

Vozlišča so `1..7`. Neobstoječo povezavo označim z ∞, diagonalo z `0`.

	1	2	3	4	5	6	7
1	0	15	68	∞	∞	84	49
2	15	0	99	30	∞	∞	∞
3	68	99	0	93	72	∞	∞
4	∞	30	93	0	79	∞	38
5	∞	∞	72	79	0	88	∞
6	84	∞	∞	∞	88	0	90
7	49	∞	∞	38	∞	90	0

B) Primov algoritem iz vozlišča 1

Koraki:

1. Start `1`.
2. Najcenejša povezava iz `1`: `1-2 (15)`.
3. Iz množice `{1,2}` dodamo `2-4 (30)`.
4. Iz `{1,2,4}` dodamo `4-7 (38)`.
5. Iz `{1,2,4,7}` dodamo `1-3 (68)`.
6. Iz `{1,2,3,4,7}` dodamo `3-5 (72)`.
7. Iz `{1,2,3,4,5,7}` dodamo `1-6 (84)`.

MST povezave:

`{1-2, 2-4, 4-7, 1-3, 3-5, 1-6}`

Skupna cena:

`15 + 30 + 38 + 68 + 72 + 84 = 307`.

Vrstni red obiskovanja: `1, 2, 4, 7, 3, 5, 6`.

C) Ali je rešitev enolična

Da. Vse uteži na povezavah so različne, zato je minimalno vpeto drevo enolično.

D) Učinkovita implementacija Prima

Graf naj bo predstavljen s seznamom sosedov, množica kandidatov pa s prioritetno vrsto oziroma min-kopico. Tako hitro dobimo najcenejšo naslednjo povezavo.

`IMG_5395.jpg` - preskočni seznam in Bellman-Ford

2. naloga - preskočni seznam

a) Uporaba generatorja naključnih števil

Pri vstavljanju elementa določimo njegovo višino z meti kovanca:

- element je vedno na nivoju 0,
- dokler generator vrača `C`, element promoviramo za en nivo,
- ko vrne `M`, se ustavimo.

Predpostavka: `C = uspeh/promocija`, `M = stop`.

b) Trditve

Pričakovano iskanje in vstavljanje v preskočni seznam sta $\Theta(\log n)$.

- $i(n) \in \Omega(n^2)$ - NE. $\log n$ ne raste vsaj tako hitro kot n^2.
- $v(n) \in O(n)$ - DA. Ker $\log n \in O(n)$.

c) Vstavljanje `7, 10, 9, 3, 5, 8, 6`

Predpostavka za zaporedje metov: `C C M M C M M M C C M C M ...`

Višine:

- 7: `C,C,M` -> nivoji 0,1,2
- 10: `M` -> nivo 0
- 9: `C,M` -> nivoji 0,1
- 3: `M` -> nivo 0
- 5: `M` -> nivo 0
- 8: `C,C,M` -> nivoji 0,1,2
- 6: `C,M` -> nivoji 0,1

Končni preskočni seznam:

```
nivo 2: 7 -------- 8
nivo 1: 6 -- 7 -- 8 -- 9
nivo 0: 3 -- 5 -- 6 -- 7 -- 8 -- 9 -- 10
```

5. naloga - Bellman-Ford

Prebrane povezave:

- `1->2 (3)`, `1->3 (1)`, `1->6 (4)`
- `2->4 (1)`, `2->6 (7)`
- `3->5 (1)`, `3->6 (3)`
- `4->5 (3)`, `4->6 (5)`
- `5->6 (1)`

Iščemo razdalje od vozlišč `1..6` do vozlišča `6`.

Najkrajše poti:

- `d_6 = 0`
- `d_5 = 1` po `5->6`
- `d_4 = 4` po `4->5->6`
- `d_3 = 2` po `3->5->6`
- `d_2 = 5` po `2->4->5->6`
- `d_1 = 3` po `1->3->5->6`

Vektor:

```
d = [3, 5, 2, 4, 1, 0]
```

PXL_20230619_080959115.jpg - preskočni seznam

b) Trditve

Pričakovano iskanje in vstavljanje sta $\Theta(\log n)$.

- $i(n) \in \Omega(n \log n)$ - NE.
- $v(n) \in O(n)$ - DA.

c) Vstavljanje `9, 7, 4, 1, 3, 5, 2, 8`

Predpostavka: `C` promovira, `M` ustavi. Prebrano zaporedje: `M C C C M M C M M M C C M C M ...`.

Višine:

- 9: `M` -> nivo 0
- 7: `C`, `C`, `C`, `M` -> nivoji 0,1,2,3
- 4: `M` -> nivo 0
- 1: `C`, `M` -> nivoji 0,1
- 3: `M` -> nivo 0
- 5: `M` -> nivo 0
- 2: `C`, `C`, `M` -> nivoji 0,1,2
- 8: `C`, `M` -> nivoji 0,1

Končni seznam:

```
nivo 3: 7
nivo 2: 2 -------- 7
nivo 1: 1 -- 2 -- 7 -- 8
nivo 0: 1 -- 2 -- 3 -- 4 -- 5 -- 7 -- 8 -- 9
```

IMG_5392.jpg - verižno množenje matrik

1) Trditve

a) "Skupno število operacij ni odvisno od postavitve oklepajev."

Odgovor: NE. Množenje matrik je asociativno glede rezultata, ne glede cene. Različni oklepaji lahko pomenijo zelo različna števila skalarnih množenj.

b) "Velikost matrike X=A_1...A_k ni odvisna od postavitve oklepajev."

Odgovor: DA, če je produkt veljaven. Končna dimenzija je določena s prvo vrstico prve matrike in zadnjim stolpcem zadnje matrike.

2) Matrike

Dimenzije:

- A = 4x6
- B = 3x8
- C = 6x3
- D = 8x2

a) Kateri vrstni red je pravilen

Možnosti:

- ABCD: ne, ker A(4x6) in B(3x8) nista združljivi.
- BDAC: ne, ker po BD dobimo 3x2, tega ne moremo množiti z A(4x6).
- ACBD: da. A*C = 4x3, potem *B = 4x8, potem *D = 4x2.
- CBAD: ne, ker po CB dobimo 6x8, tega ne moremo množiti z A(4x6).

Pravilno: ACBD.

b) Najmanjše število produktov

Za verigo A C B D so dimenzije:

```
4, 6, 3, 8, 2
```

Najboljša postavitev oklepajev:

```
A * (C * (B * D))
```

Cena:

- B*D: 3*8*2 = 48, rezultat 3x2
- C*(BD): 6*3*2 = 36, rezultat 6x2
- A*(C(BD)): 4*6*2 = 48, rezultat 4x2

Skupaj:

$48 + 36 + 48 = 132$.

20240620_100533.jpg - nahrbtnik

Podatki:

predmet	1	2	3	4	5
velikost	2	5	3	8	1
vrednost	3	5	5	9	3

Kapaciteta: $V=8$.

a) Dovoljeno rezanje predmetov

Razmerja v/w:

- predmet 1: $3/2 = 1.5$
- predmet 2: $5/5 = 1$
- predmet 3: $5/3 \approx 1.67$
- predmet 4: $9/8 = 1.125$
- predmet 5: $3/1 = 3$

Vrstni red: $5, 3, 1, 4, 2$.

Vzamemo:

- predmet 5 v celoti: velikost 1, vrednost 3
- predmet 3 v celoti: velikost 3, vrednost 5
- predmet 1 v celoti: velikost 2, vrednost 3
- ostane kapaciteta 2, vzamemo $2/8$ predmeta 4: vrednost $9*(2/8)=2.25$

Skupna vrednost:

$3 + 5 + 3 + 2.25 = 13.25$.

b) 0/1 nahrbtnik

Najboljša vrednost je 11.

Optimalni izbiri:

- predmeti $\{1, 3, 5\}$: velikost $2+3+1=6$, vrednost $3+5+3=11$
- ali $\{1, 2, 5\}$: velikost $2+5+1=8$, vrednost $3+5+3=11$

Preverjanje: predmet 4 sam da vrednost 9, kombinacija 2+3 ima vrednost 10, zato 11 ostane najboljše.

20240620_100530.jpg - zgoščena tabela

Zaporedje: $ALGORITMI$.

$h(k) = (k*p) \bmod m$, $p=11$.

a) Veriženje, elementi v istem košu kot BST

Za $m=5$ velja $11 \bmod 5 = 1$, zato je $h(k)=k \bmod 5$.

ASCII ostanki:

- $A=65 \to 0$
- $L=76 \to 1$
- $G=71 \to 1$
- $O=79 \to 4$
- $R=82 \to 2$
- $I=73 \to 3$
- $T=84 \to 4$
- $M=77 \to 2$
- $I=73 \to 3$

Končno:

- koš 0: A
- koš 1: BST z L koren, G levo
- koš 2: BST z R koren, M levo
- koš 3: I oziroma pri dovoljenih duplikatih I z drugim I
- koš 4: BST z O koren, T desno

Opomba: če struktura predstavlja slovar, drugi I posodobi obstoječ ključ. Če naloga obravnava črke kot zaporedne vnose, se drugi I shrani kot duplikat po dogovorjenem pravilu.

b) Linearno prenaslavljanje, ob polni tabeli podvojimo velikost

Najprej $m=5$:

- $A \to 0$
- $L \to 1$
- $G \to 1$, zasedeno, gre na 2
- $O \to 4$
- $R \to 2$, zasedeno, gre na 3

Tabela je polna. Pred vstavljanjem I povečamo na $m=10$ in rehashamo:

- $A=65 \to 5$
- $L=76 \to 6$
- $G=71 \to 1$
- $O=79 \to 9$
- $R=82 \to 2$

Nadaljujemo:

- $I=73 \to 3$
- $T=84 \to 4$

- `M=77 -> 7`
- drugi `I=73`: če dovolimo duplikat, linearno išče `3,4,5,6,7,8` in gre na `8`; če gre za slovar, posodobi obstoječ `I` na indeksu 3.

Končna tabela pri dovoljenih duplikatih:

indeks	0	1	2	3	4	5	6	7	8	9
vrednost	-	G	R	I	T	A	L	M	I	O

`20240620_100452.jpg` - višinska mapa

Naloga išče najcenejšo pot od celice 0 do 24 po 5x5 mreži z dovoljenimi premiki naravnost in diagonalno.

Prebrana višinska tabela:

```
0:7   1:0   2:3   3:2   4:5
5:3   6:3   7:4   8:1   9:1
10:2  11:7  12:5  13:5  14:6
15:3  16:0  17:5  18:5  19:5
20:7  21:5  22:1  23:6  24:4
```

Pomembna nejasnost: fotografija pravi, da je pri spustu cena negativna, kar lahko ustvari negativne cikle. Če so cikli dovoljeni, najcenejša pot ni dobro definirana, ker lahko s ponavljanjem cikla ceno poljubno zmanjšamo.

Zato sta smiselni dve interpretaciji:

Interpretacija 1 - pot brez ponavljanja vozlišč

Uporabimo Bellman-Ford samo kot idejo sproščanja, vendar moramo prepovedati ponavljanje vozlišč. Zaradi negativnih spustov je treba v rešitvi izrecno paziti na cikle. Natančnega enoličnega rezultata iz slike ne morem varno potrditi brez jasnejšega pravila o ciklih.

Interpretacija 2 - spust ima pozitivno ceno absolutne višinske razlike

Če spust štejemo kot pozitivni napor, dobimo najcenejšo pot:

`0 -> 6 -> 7 -> 13 -> 18 -> 24`

Cena:

- `0->6`: diagonala, `|3-7|=4`, cena `0.7*4 = 2.8`
- `6->7`: vzpon za 1, cena `1^2 = 1`
- `7->13`: diagonala, vzpon za 1, cena `0.7*1^2 = 0.7`
- `13->18`: enaka višina, cena `0`
- `18->24`: diagonala, spust za 1, cena `0.7*1 = 0.7`

Skupaj: `5.2`.

`20240610_100725.jpg` - rdeče-črna drevesa

a) DA/NE

A) "Rdeče-črno drevo je dvojiško drevo."

DA. Rdeče-črno drevo je dvojiško iskalno drevo z dodatnimi barvnimi pravili.

B) "Če RČ drevo vsebuje n elementov, bo iskanje potrebovalo `O(n^2)` časa."

DA kot zelo ohlapna zgornja meja, vendar nenatančno. Pravilna tesna meja je `O(log n)`, ker je višina RČ drevesa `O(log n)`.

C) "Če RČ in AVL drevo vsebujeta iste elemente, bo globina RČ drevesa večja od globine AVL drevesa."

NE. AVL je strožje uravnotežen, zato ima pogosto manjšo višino, ni pa nujno za vsako konkretno množico in obliko drevesa, da je RČ strogo globlje.

D) "Najmanjši element v AVL drevesu je vedno v korenu."

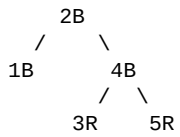
NE. V BST/AVL je najmanjši element skrajno levo.

E) "Število rdečih elementov v RČ drevesu je za ena manjše od števila črnih."

NE. Pravila določajo črno višino in prepoved dveh zaporednih rdečih, ne fiksnega razmerja rdečih in črnih vozlišč.

b) Vstavimo 9, 8, 7 v dano RČ drevo

Začetek:



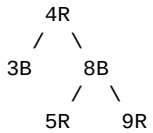
Vstavi 9:

- 9 gre desno od 5.
- Nov element je rdeč.
- Starš 5 je rdeč, stric 3 je rdeč -> prebarvanje: 3 in 5 postaneta črna, 4 postane rdeč.
- Koren 2 ostane črn.

Vstavi 8:

- 8 gre levo od 9.
- Starš 9 je rdeč, stric je črn/prazen.
- Primer RL glede na 5: najprej desna rotacija pri 9, nato leva rotacija pri 5, zamenjava barv.

Desno poddrevo pri 4 postane približno:



Vstavi 7:

- 7 gre desno od 5.
- Starš 5 je rdeč, stric 9 je rdeč -> prebarvanje: 5 in 9 črna, 8 rdeča.
- Ker je 8 otrok rdečega 4, popravljamo še pri 4/8 glede na koren 2.

Končna oblika je odvisna od uporabljene konvencije NIL-listov in zaporedja popravljanja, ključni postopki pa so: prebarvanje pri rdečem stricu, nato rotacija/prebarvanje pri dveh zaporednih rdečih vozliščih.

`PXL_20230619_081022358.jpg` - drevo

Fotografija je delno odrezana, zato je del B rekonstruiran po vidnem drevesu.

A) Trditve

Polno dvojiško iskalno drevo `T` globine `h`; odstranimo vsak drug list z leve proti desni in dobimo `T'`.

a) "Vsako polno dvojiško drevo je AVL drevo."

DA, če "polno" pomeni popolno drevo, kjer so vsi nivoji zapolnjeni. Višini levega in desnega poddrevesa sta povsod enaki.

b) Če je `t(n)` število listov drevesa `T`, potem `t(n) ∈ O(n^2)`.

DA. Velja celo `t(n) <= n`, zato je tudi `O(n^2)`.

c) "`T'` je AVL drevo."

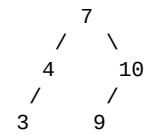
DA po interpretaciji, da odstranimo izmenično liste tako, da vsak starš na zadnjem nivoju izgubi natanko enega otroka. Višinska razlika ostane največ 1.

d) "Število listov drevesa `T'` je `2^(h-1)-1`."

NE. Če ima popolno drevo globine `h` `2^h` listov, po odstranitvi vsakega drugega ostane `2^(h-1)` listov.

B) Vstavljanje 2 in 8 v AVL

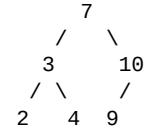
Vidno začetno drevo:



Vstavimo 2:

- 2 gre levo od 3.
- Vozlišče 4 postane neuravnoteženo tipa LL.
- Naredimo desno rotacijo pri 4.

Po vstavljanju 2:

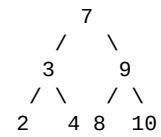


Vstavimo 8:

- 8 gre levo od 9.
- Vozlišče 10 postane neuravnoteženo tipa LL.

- Naredimo desno rotacijo pri 10.

Končno:



Super učni vodič

Super učni vodič APS2

To je glavni “lock-in” dokument. Beri ga aktivno: pri vsakem algoritmu si na papir naredi en mini primer.

1. Asimptotika in ocenjevanje časa

Cilj

Znati oceniti, kako hitro raste čas algoritma, in iz tega sklepati, ali je algoritem uporaben za velik n.

Kdaj uporabim

Ko naloga sprašuje po O, Θ, Ω, primerjavi algoritmov ali oceni časa pri danem računalniku.

Postopek

1. Prepoznaj osnovno obliko: zanke, rekurzija, število povezav v grafu, velikost tabele.
2. Ignoriraj konstante in nižje člene.
3. Če moraš oceniti sekunde: uporabi $T(n)=C*f(n)$ in deli s številom operacij/s.

Mini primer

$n = 10^6$, algoritem $O(n^2)$: približno 10^{12} operacij. Pri 10^9 operacij/s je to približno $1000\ s$, torej okrog 17 minut.

Tipične napake

- Zamenjaš zahtevnost algoritma z zahtevnostjo problema.
- Pri polnem grafu pozabiš, da je $m=\Theta(n^2)$.
- Pri $n \log n$ uporabiš občutek kot da je skoraj n; ni isto, ampak je še vedno veliko boljše kot n^2.

Preverjanje

Če se pri velikem n kvadratični algoritem zdi “čisto hiter”, si skoraj gotovo zgrešil faktor.

2. Amortizirana analiza

Cilj

Dokazati, da je zaporedje operacij poceni v povprečju, čeprav je posamezna operacija včasih draga.

Kdaj uporabim

Dinamična tabela, razširjen sklad, dvojiški števec, tabela tabel, strukture s periodičnim preurejanjem.

Postopek

1. Ne glej samo ene operacije.

2. Preštej skupno delo čez n operacij.
3. Deli z n.

Mini primer: dvojiški števec

Bit 0 se spremeni n-krat, bit 1 $n/2$-krat, bit 2 $n/4$-krat ...

Skupaj $< 2n$, zato je `povecaj()` amortizirano $O(1)$.

Tipične napake

- Rečeš “najslabši primer je $O(k)$, zato je tudi amortizirano $O(k)$” - lahko je, ampak pogosto ni tesno.
- Pri potencialu izbereš funkcijo, ki gre v dolg.

Preverjanje

Vprašaj se: “Ali lahko vsak drag dogodek zaračunam nečemu, kar se zgodi omejeno velikokrat?”

3. Tabele, slovarji, zgoščevanje

Zgoščena tabela

Cilj

Hitro vstavljanje, iskanje in brisanje po ključu.

Kdaj uporabim

Ko imamo ključe in želimo pričakovano $O(1)$ dostop.

Postopek

1. Izračunaj $h(k)$.
2. Če je veriženje: vstavi v koš.
3. Če je odprto naslavljanje: išči naslednje mesto z $h(k,i)$.
4. Spremljaj faktor zasedenosti $\alpha=n/m$.

Mini primer

$h(k)=k \bmod 5$, ključ 17: $17 \bmod 5 = 2$, gre v koš 2.

Pazi

- Pri linearnem naslavljanju nastane grupiranje.
- Pri brisanju v odprtem naslavljanju ne smeš samo izprazniti celice; uporabiš `DELETED`.
- Pri duplikatih ključa slovar običajno posodobi vrednost.

Bloomov filter

Cilj

Zelo prostorsko poceni test pripadnosti.

Odgovori

- “Zagotovo NE”
- “Verjetno DA”

Pazi

False positive je možen. False negative ni možen, razen če nepravilno brišeš.

4. Drevesa

BST

Kdaj uporabim

Urejeni ključi, iskanje minimuma/maksimuma, predhodnik/naslednik.

Koraki iskanja

1. Primerjaj s korenom.
2. Manjše gre levo, večje desno.
3. Ponavljaj do zadetka ali praznega otroka.

Zahtevnost

`O(h)`, kjer je h višina. Uravnoteženo: `O(log n)`, degenerirano: `O(n)`.

AVL

Cilj

BST, ki po vsaki spremembi ostane uravnotežen.

Pravilo

Za vsako vozlišče: `|višina(levo)-višina(desno)| <= 1`.

Rotacije

- LL -> desna rotacija.
- RR -> leva rotacija.
- LR -> leva pri levem otroku, nato desna.
- RL -> desna pri desnem otroku, nato leva.

Mini trik

Najprej najdi najnižje neuravnoteženo vozlišče. Tam odloči tip rotacije.

Rdeče-črno drevo

Kdaj uporabim

Ko želiš `O(log n)` in manj strogo uravnoteženje kot AVL.

Pravila

- Koren je črn.
- Rdeč nima rdečega otroka.
- Vse poti do praznih listov imajo enako črno višino.

Pazi

Novo vozlišče vstaviš kot rdeče. Če je starš črn, si končal. Če je starš rdeč, gledaš strica:

- rdeč stric -> prebarvanje,
- črn stric -> rotacija + prebarvanje.

5. Kopice in prioritetna vrsta

Cilj

Hitro dobiti najmanjši/največji element.

Min-kopica

- `min`: `O(1)`
- `insert`: `O(log n)`
- `deleteMin`: `O(log n)`
- `heapify` celotne tabele: `O(n)`

Tipične napake

- Najmanjši element v min-kopici je v korenu, največji je nekje med listi.
- Kopica ni BST. Le lokalni odnos starš-otrok je urejen.

6. Disjunktne množice

Cilj

Hitro preverjati, ali sta elementa v isti komponenti.

Operacije

- `makeSet(x)`
- `find(x)`
- `union(x,y)`

Kdaj uporabim

Kruskal, povezane komponente, združevanje skupin.

Trik

Vedno uporabi:

- union by rank/size,
- path compression.

Potem je amortizirano skoraj `O(1)`.

7. Dinamično programiranje

Cilj

Problem razbiješ na prekrivajoče se podprobleme in rezultate shraniš.

Kako prepoznaš DP

- “Največja/najmanjša vrednost”
- “Na koliko načinov”
- “Optimalna postavitev”
- “Dovoljene odločitve po korakih”

Postopek

1. Definiraj stanje `dp[...]`.
2. Zapiši prehod.
3. Zapiši osnovne primere.
4. Izberi vrstni red računanja.
5. Iz rezultata po potrebi rekonstruiraj izbire.

Stopnice

`f(n)=f(n-1)+f(n-2)`, če so koraki 1 ali 2.

Osnovi: `f(0)=1`, `f(1)=1`.

0/1 nahrbtnik

`dp[i][w]` = najboljša vrednost s prvimi `i` predmeti in kapaciteto `w`.

Prehod:

```
dp[i][w] = max(dp[i-1][w], dp[i-1][w-w_i] + v_i)
```

Če `w_i > w`, predmeta ne moreš vzeti.

Delni nahrbtnik

Če smeš rezati predmete, ni DP. Uporabi požrešno:

1. Izračunaj `v_i/w_i`.
2. Uredi padajoče.
3. Jemlji najboljše razmerje, zadnji predmet po potrebi odreži.

Verižno množenje matrik

`c(i,j)=min(c(i,k)+c(k+1,j)+d_{i-1}d_kd_j)`

Računaš po diagonalah. Rezultat je `c(1,n)`.

8. Grafi - samo zahtevani del

Prim

Kdaj

MST, povezan neusmerjen utežen graf.

Koraki

1. Začni v vozlišču.
2. Množico drevesa širi z najcenejšo povezavo navzven.
3. Ustavi se pri `n-1` povezavah.

Preverjanje

MST ima vedno `n-1` povezav in poveže vsa vozlišča brez cikla.

Kruskal

Koraki

1. Uredi povezave po teži.
2. Dodajaj najcenejše, ki ne naredijo cikla.
3. Cikle preverjaj z disjunktnimi množicami.

Pazi

Če ima graf več komponent, dobiš minimalni vpeti gozd, ne drevesa.

Topološko urejanje

Kdaj

Samo DAG.

Koraki

1. Vhodne stopnje.
2. V vrsto vozlišča z vhodno stopnjo 0.
3. Odstranjuj in zmanjšuj stopnje.
4. Če ostane vozlišče, ima graf cikel.

Dijkstra

Kdaj

Najkrajše poti z nenegativnimi utežmi.

Pazi

Negativna utež -> Dijkstra odpade.

Bellman-Ford

Kdaj

Negativne uteži, zaznava negativnega cikla.

Koraki

1. Inicializiraj razdalje.
2. `n-1` krat sprosti vse povezave.
3. Dodatni prehod zazna negativen cikel.

Welsh-Powell

Kdaj

Hitro barvanje grafa.

Koraki

1. Uredi vozlišča padajoče po stopnji.
2. Vzemi prvo nepobarvano, dodeli novo barvo.
3. Z isto barvo pobarvaj nesosednja vozlišča.
4. Ponavljaj.

Pazi

Ni nujno optimalen.

9. Kompleksnost P/NP

P

Odločitveni problemi, rešljivi v polinomskem času.

NP

Problemi, pri katerih lahko dano rešitev preverimo v polinomskem času.

NP-poln

Problem je v NP in vsak NP problem se nanj polinomsko reducira.

NP-težak

Vsaj tako težak kot NP-polni problemi, ni nujno v NP.

Tipična izpitna past

Če ima problem znan eksponenten algoritem, to ne pomeni, da problem ni v P. Pomeni samo, da s tem algoritmom pokažeš zgornjo mejo.

10. Najbolj tipične izpitne naloge

“DA/NE in utemelji”

Piši kratko:

1. DA/NE.
2. En stavek definicije.
3. Protiprimer, če je NE.

“Izvedi algoritem na grafu”

Vedno pokaži tabelo korakov:

- trenutna množica,
- kandidati,
- izbrana povezava/vozlišče,
- posodobljene razdalje/barve.

“DP naloga”

Ne začni s kodo. Najprej stanje in rekurenca. Brez tega rešitev izgleda kot ugibanje.

“Drevo po vstavljanju”

Po vsakem vstavljanju preveri najnižji neuravnotežen prednik. Ne popravljaj od korena navzdol.

Izpitni povzetek A4

Izpitni povzetek A4

Asimptotika

- O: zgornja meja, Ω: spodnja meja, Θ: tesna meja.
- Poln graf: $m=\Theta(n^2)$.
- 1 GHz ≈ 10^9 operacij/s.
- $\log n < n < n \log n < n^2 < n^3 < 2^n < n!$

Amortizirana analiza

- Metoda vsote: skupno delo $T(n)$ -> amortizirano $T(n)/n$.
- Dvojiški števec: $n+n/2+n/4+...<2n$ -> $O(1)$ amortizirano.
- Dinamična tabela s podvajanjem -> $\texttt{insert}\ O(1)$ amortizirano.
- Potencial: $a_i = c_i + \Phi(D_i)-\Phi(D_{i-1})$.

Zgoščevanje

- Veriženje: pričakovano $O(1+\alpha)$, $\alpha=n/m$.
- Linearno: $h(k,i)=(h(k)+i) \bmod m$.
- Brisanje pri odprtem naslavljanju: `DELETED`, ne prazno.
- Bloom: “zagotovo NE” / “verjetno DA”; false positive da, false negative ne.

Drevesa

- BST: $\texttt{search/insert/delete} = O(h)$.
- Minimum BST: skrajno levo; maksimum: skrajno desno.
- AVL: $|h_L-h_R|\leq 1$.
- Rotacije: LL desna, RR leva, LR leva+desna, RL desna+leva.
- RČ: koren črn, rdeč nima rdečega otroka, vse poti imajo isto črno višino.
- AVL bolj uravnotežen; RČ manj rotacij.

Kopica

- Min-kopica: $\texttt{min}\ O(1)$, $\texttt{insert}\ O(\log n)$, $\texttt{deleteMin}\ O(\log n)$, $\texttt{heapify}\ O(n)$.
- Kopica ni BST.

Disjunktne množice

- `makeSet`, `find`, `union`.
- Uporabi path compression + union by rank/size.
- Kruskal: DSU za preverjanje ciklov.

DP

- Koraki: stanje -> prehod -> osnove -> vrstni red -> rekonstrukcija.
- Stopnice 1/2: `f(n)=f(n-1)+f(n-2)`, `f(0)=1`, `f(1)=1`.
- 0/1 nahrbtnik: `dp[i][w]=max(dp[i-1][w], dp[i-1][w-w_i]+v_i)`.
- Delni nahrbtnik: sort po `v_i/w_i`, jemlji pohlepno.
- Matrična veriga: `c(i,j)=min_k(c(i,k)+c(k+1,j)+d_{i-1}d_kd_j)`, čas `O(n^3)`.

Grafi

- MST ima `n-1` povezav, brez cikla, poveže vsa vozlišča.
- Prim: širi drevo z najcenejšo povezavo navzven.
- Kruskal: sort povezav, dodaj brez cikla, DSU.
- Topološko: samo DAG; vhodne stopnje 0 -> odstranjuj.
- DAG shortest path: topološki vrstni red + sproščanje, `O(n+m)`.
- Dijkstra: samo nenegativne uteži.
- Bellman-Ford: negativne uteži OK; `n-1` relaksacij + 1 za negativen cikel.
- Welsh-Powell: uredi vozlišča po padajoči stopnji, barvaj nesosednja z isto barvo.
- Največji pretok: preskoči.

P/NP

- P: rešljivo v polinomskem času.
- NP: rešitev preverljiva v polinomskem času.
- NP-poln: v NP + vsi NP problemi se reducirajo nanj.
- NP-težak: vsaj tako težak kot NP-polni, ni nujno v NP.
- Algoritem ≠ problem. Slab algoritem ne dokazuje, da problem ni v P.

Mini pasti

- Negativna utež -> ne Dijkstra.
- Različne uteži v grafu -> MST enoličen.
- Slovar z istim ključem tipično posodobi vrednost.
- Pri matrix-chain rezultat dimenzije ni odvisen od oklepajev, cena pa je.
- Pri `O(n)` velja tudi `O(n^2)`, ampak to ni tesna meja.

Status

Status priprave gradiva

Uspešno narejeno

- Pregledana je struktura map `Predavanja`, `Vaje`, `Primeri`.
- Ustvarjena je mapa `results`.
- Pripravljen je inventar datotek.
- Predavanja so obdelana v spremljevalni datoteki z dopolnitvami in rešitvami praznih vprašanj.
- Vaje so uporabljene kot referenca za tipe nalog: asimptotika, amortizacija, tabele, drevesa, zgoščevanje, DP, grafi.
- Slike iz `Primeri` so pregledane in rešene, z označenimi nejasnostmi.
- Pripravljen je glavni učni vodič.
- Pripravljen je zgoščen A4 izpitni povzetek.

Nastale datoteke

- `results/inventar.md`
- `results/resitve_predavanja.md`
- `results/resitve_primeri.md`
- `results/super_ucni_vodic.md`
- `results/izpitni_povzetek_A4.md`
- `results/status.md`
- `results/primeri_kontaktni_list.jpg` - pomožni kontaktni list slik

Nejasnosti

- Nekatere slike so fotografirane pod kotom ali delno odrezane, zato so določene rešitve označene s predpostavko.
- Pri višinski mapi iz `20240620_100452.jpg` formula za spust ni popolnoma nedvoumna. Če spust dobesedno štejemo kot negativno ceno, se lahko pojavijo negativni cikli; zato sem zapisal tudi varnejšo interpretacijo.
- Pri zgoščeni tabeli z besedo `ALGORITMI` je druga črka `I` lahko obravnavana kot duplikat ali kot posodobitev istega ključa; zapisani sta obe možnosti.
- Pri nekaj starejših PXL slikah je spodnji del naloge delno zakrit, zato so rešitve omejene na jasno berljiv del.

Teme, ki jih najbolj ponovi

1. Grafi: Prim, Kruskal, Dijkstra, Bellman-Ford, topološko urejanje, Welsh-Powell.
2. Dinamično programiranje: nahrbtnik, stopnice, verižno množenje matrik.
3. Drevesa: AVL rotacije, RČ pravila, BST minimum/maksimum.
4. Zgoščevanje: veriženje, odprto naslavljanje, resize, Bloomov filter.
5. Amortizirana analiza: dinamična tabela, sklad, dvojiški števec.
6. Asimptotika in P/NP definicije.

Predlagan vrstni red učenja do izpita

1. Najprej preberi `izpitni_povzetek_A4.md`, da dobiš zemljevid snovi.
2. Nato predelaj `super_ucni_vodic.md` po sklopih in si ob vsakem naredi 1 mini primer.
3. Reši grafe iz `resitve_primeri.md` še enkrat brez gledanja rešitve.
4. Reši DP primere: nahrbtnik, stopnice, matrix-chain.
5. Na koncu preleti `resitve_predavanja.md`, posebej prazna mesta in “pazi” opombe.

Največ točk se običajno izgubi pri postopku, ne pri končnem številu. Zato na izpitu vedno pokaži korake.